

KosmoOrbit — Technik & Bau

Die Architektur-Designzentrale des Baubüros ArchitekturKosmos. Ein lokal-first Monorepo für Architektur/BIM mit steuernder Büro-KI «Kosmo» — was darin eingekaufte Technologie ist, was Eigenbau, welche Funktionen es gibt, und wie es gebaut wurde.

8

Workspaces (7 Pakete + App)

~102

Commands = Kosmos Können

~60

pure Derive-Funktionen

765/847

Kernel-/App-Tests grün

In einem Satz: Alles ist ein *Command* mit einem zod-Schema; jedes Schema wird automatisch ein Kosmo-LLM-Werkzeug. Ein Command gibt Patches zurück — daraus fließen Undo, Yjs-Sync und die `.kosmo`-Pakete. Alle Darstellungen (Plan, Schnitt, 3D, Mengen) sind *pure Ableitungen* aus dem Dokument, nie umgekehrt.

Was dieses Dokument abdeckt

- Kap. 1-2 · Überblick & Monorepo-Landkarte
- Kap. 3-4 · Dritt-Technologien (eingekauft) & Sprachen/Build
- Kap. 5-7 · Eigenbau (selbst gecodet), KI-Schicht, die 9 Stationen
- Kap. 8 · Qualität & CI/Packaging
- Kap. 9 · Bau-Story — wie es tatsächlich entstanden ist (Agenten-Orchestrierung, Golden-Disziplin, Ehrlichkeitsregel)

KosmoOrbit ist ein **lokal-first Monorepo** (npm workspaces, Node ≥ 22) für den kompletten Architektur-Workflow eines Schweizer Baubüros: BIM-Kern, 2D-Pläne (SIA-konform), 3D-Visualisierung, Wissens-/Datenverwaltung und eine steuernde Büro-KI. «Lokal-first» heisst: das Modell und die Daten leben auf dem Rechner der Nutzerin; Cloud/HomeStation sind optional und werden im UI ehrlich benannt, nie vorgetäuscht.

Das Grundprinzip: «alles ist ein Command»

Jede schreibende Aktion — eine Wand zeichnen, ein Geschoss kopieren, ein Blatt platzieren — ist ein registriertes **Command** mit einem `zod`-Parameter-Schema und einer reinen `run()`-Funktion, die *Patches* zurückgibt. Aus diesem einen Mechanismus fällt alles Weitere ab:

- **Undo** = die Inverse der Patches (`invertPatches`).
- **Live-Sync** zwischen Geräten = dieselben Patches durch Yjs/CRDT.
- **KI-Steuerung** = jedes Command-Schema wird automatisch ein LLM-Werkzeug. «Was Kosmo kann, ist genau die Menge der Commands.»
- **Kosmo-Vorschläge** = Diff-Karten, die beim «Anwenden» durch *denselben* Command-Weg laufen — atomare, umkehrbare Undo-Gruppen.

Alle **Darstellungen** (Grundriss, Schnitt, Axonometrie, Mengen, 3D-Szene, Render-Graph) entstehen als **pure Funktionen** aus dem Dokument (`derive/`). Das Dokument ist die einzige Wahrheit; Bilder, GLB-Daten und Job-Status leben bewusst in Laufzeit-Stores ausserhalb des Modells.

MONOREPO-LANDKARTE

Sieben Bibliotheks-Pakete unter `packages/`, eine App unter `apps/`, dazu Hilfs-Server/-Skripte unter `tools/`.

PAKET	VERSION	ZWECK
@kosmo/kernel	1.0.0-v1	BIM-Kern: Entities, Commands (Undo/Journal), Geometrie, gesamte Derive-Pipeline — pure TS, läuft im Web-Worker
@kosmo/ai	1.0.0-v1	KI-Schicht: Provider, Tool-Registry aus Commands, Personas, gated Ausführung
@kosmo/contracts	1.0.0-v1	Geteilte zod-Verträge zur HomeStation (render-scene/v1, Bridge-API, .kosmo-Paket)
@kosmo/data	1.0.0-v1	CH-Bauteil- und Materialkatalog (KosmoData)
@kosmo/sync	1.0.0-v1	Yjs-CRDT-Client (entity-genaues LWW, Offline-Warteschlange), host-agnostisch
@kosmo/ui	1.0.0-v1	Design-System «Aura»: Tokens, Themes, Logo, Icons, Basiskomponenten
@kosmo/lizenz	1.0.0-v1	Signierte Lizenz — reine Ed25519-Verify-Funktion (Web Crypto, keine npm-Dep)
@kosmo/orbit-app	0.7.4	Die React-App (Desktop + iPad) — Stationen, Shell, Zustand

tools/ — Server & Skripte (kein Workspace)

- `homestation-bridge/` — Python/FastAPI-Bridge zur Render-Pipeline (`--fake` im Container)
- `sync-server/` — Node/Hocuspocus-Yjs-Server + SQLite-Persistenz
- `svg-qa/pruefe-goldens.mts` — Golden-SVG-QA (rastert in echtem Chromium)
- `docling-ingest/` — Python PDF→Markdown-Wissens-Ingest
- `secret-scan.mjs` / `netz-check.mjs` — Sicherheits-Gates (reines Node)

Bewusst schlanke, etablierte Bausteine — kein LLM-SDK (Kosmo spricht mit LLMs über plain `fetch`), keine schweren Frameworks über das Nötige hinaus.

BIBLIOTHEK	VERSION	WOFÜR
React + React-DOM	^19.2	UI-Framework der App
Three.js (+ camera-controls)	0.184 / 3.1	3D-Rendering & Kamerasteuerung (direkt, ohne react-three)
Zustand	^5.0	State-Stores (src/state/*)
Zod	^4.1	Schema-Validierung — Basis der Command-Parameter
Yjs + y-indexeddb + Hocuspocus	13.6 / 9.0 / 4.3	CRDT-Live-Sync + Offline-Persistenz
Tauri (Rust)	2.x	Desktop-Verpackung (Dialog/FS/Tray)
Vite + vite-plugin-pwa	^7.0 / ^1.3	Build + Progressive-Web-App
Vitest / Playwright	^3.2 / ^1.61	Unit-/Komponenten-Tests · End-to-End
jsPDF + svg2pdf.js + pdfjs-dist	4.2 / 2.7 / 6.1	PDF-Export (Pläne/Blätter) & PDF-Anzeige/Import
web-ifc	0.0.77	IFC-Interop (BIM-Austausch)
clipper2-ts / earcut	2.0 / 3.0	Polygon-Boolean/Offset (Wände, Pochierung) · Triangulierung (Mesh)
perfect-freehand / fflate / suncalc	1.2 / 0.8 / 2.0	Freihand-Skizzen · Zip (.kosmo/GLB) · Sonnenstand/Besonnung
jsonrepair / MSAL	3.13 / 5.16	Repariert LLM-Tool-Argument-JSON · Microsoft/OneDrive-Login
FastAPI / uvicorn / httpx / faster-whisper	Python ≥3.10	HomeStation-Bridge (Render/STT/TTS)

TypeScript (überall, ^5.9)

Streng konfiguriert: `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes` (optionale Felder brauchen konditionale Spreads), `verbatimModuleSyntax`, Target ES2022 inkl. WebWorker-Libs. App-Build: `tsc --noEmit && vite build`.

Rust / Tauri 2

Desktop-Verpackung (`src-tauri/`, edition 2021): Crates `tauri` v2 (Features `tray-icon`, `image-png`), `tauri-plugin-dialog`, `tauri-plugin-fs`, `serde`. Release-Profil mit `strip/lto`. Identifier `ch.architekturkosmos.kosmoorbit`, strikte CSP, zweites Fenster «kosmo-charakter». Drei **Editionen** aus einem Codebase (nur `VITE_KOSMO_EDITION` unterscheidet): **standard** (HomePC lokal, Ollama), **remote** (dünner Client über VPN), **cloud** (voll Claude-abhängig, Opus 4.8, kein lokales LLM).

Python (HomeStation-Bridge)

FastAPI ≥ 0.115 + uvicorn + httpx; optional faster-whisper (STT) und pynacl (Ed25519). Im Container läuft sie als `--fake` (simulierte GPU) für Tests.

Node (Sync-Server)

Hocuspocus-Server + SQLite-Extension; Token- und optional Lizenz-Auth, Rate-Limit, Payload-Deckel.

Das ist der eigentliche Wert und die Eigen-IP — *nicht* generischer Glue-Code, sondern eine durchdachte, zodi-getriebene Kommando- und Ableitungs-Maschine.

Command-System — [kosmo-kernel/src/commands/](#)

Zentrale Schreib-Schnittstelle. `core.ts` definiert `Command<P>` (`id`, `params: zodSchema`, `summarize()`, `pure run()`), eine globale Registry, `registerCommand()`, `execute()` (mit `dryRun` für die Vorschau der Diff-Karten) und ein `JournalEntry` mit Actor (`benutzer/kosmo/kosmodev/...`). **~102 registrierte Commands**, verteilt auf:

DATEI	COMMANDS	DOMÄNE
<code>design.ts</code>	69	Wände, Geschosse, Öffnungen, Mesh, Dach, Treppe, Fassadenmodule, Segmentierung, Grundriss-Generator, Möbel, Regeln
<code>publish.ts</code>	20	Blätter, Ansichten platzieren, Revisionen, Baugesuch
<code>vis.ts</code>	10	Render-Graph (Nodes/Kanten)
<code>grundlagen.ts / core.ts</code>	3	Volumenstudie u.a.

Command → LLM-Werkzeug — [kosmo-ai/src/tools.ts](#)

Die Brücke, die «alles ist ein Command» mit «Kosmo kann steuern» verbindet: `commandTools()` mappt jedes Command auf eine Werkzeug-Definition, indem es das zod-Schema per `z.toJSONSchema()` in JSON-Schema übersetzt. Kein manuelles Tool-Wiring — neue Commands sind automatisch für Kosmo verfügbar.

Derive — ~60 pure Ableitungsfunktionen — [kosmo-kernel/src/derive/](#)

Alles entsteht aus dem Doc. Ausgewählte Vertreter: **Plan/2D** (`plan.ts`, `plansvg.ts`, `sheet.ts`, `poche.ts`, `hiddenline.ts`, `dimensions.ts`, `schraffur.ts`), **Schnitt** (`section.ts`), **Axo/3D** (`axo.ts`, `scene.ts`, `mesh.ts`, `gltf.ts`, `kamera.ts`), **Mengen/Kosten** (`mengen.ts`, `ausmass.ts`, `kostenschaetzung.ts`, `sia416.ts`), **Vis** (`visgraph.ts`, `renderprompt.ts`) sowie Fachdomänen wie `baugesuch.ts`, `ausnuezungsnachweis.ts`, `schwarzplan.ts`, `besonnungsvergleich.ts`, `beschlag.ts` (neu in 0.7.4).

Patch/Undo-Modell & Golden-Tests

`model/doc.ts` definiert `AnyPatch`, `invertPatches()` (Undo) und `apply()`. Aus denselben Patches fließen Yjs-Sync und `.kosmo`-Pakete. Die **28 Golden-SVGs** (`test/golden/`) müssen byte-identisch bleiben, wo ein neues Feature ohne Daten inaktiv ist — die härteste Regressionswache des Projekts.

`packages/kosmo-ai/` — die Büro-KI «Kosmo», provider-neutral.

- **Provider-Abstraktion** (`provider.ts`): ein `ChatProvider` als async-iterable Stream. Konkret: `OllamaProvider`, `AnthropicProvider`, `OpenAiKompatibelProvider` (LM Studio), `MockProvider` und `ScriptedProvider` (deterministische E2E). Alle über plain `fetch` — bewusst kein SDK.
- **ChatSession** (`chat.ts`): die Gesprächsschleife mit Tool-Aufruf-Orchestrierung und *gated* Ausführung — schreibende Vorschläge werden erst Diff-Karten, laufen nach Freigabe durch `runCommand`.
- **Personas** (`personas.ts`): `kosmo` / `kosmodev` / `kosmodoc` / `kosmotrain` mit CH-deutschen Systemprompts; Kosmo gibt sich in der Cloud-Betriebsart nie als Basismodell aus, antwortet aber ehrlich, wenn direkt gefragt.
- **Vision** (`bild-budget.ts`): Kosmo kann Screenshots «sehen» (Bild-Token-Budget).

DIE NEUN STATIONEN

`apps/kosmo-orbit/src/modules/*` — die App gliedert sich in neun Arbeitsstationen, navigiert über das Orbit-Startmenü und das Boden-Dock.

STATION	WAS SIE TUT
design	Entwurf/BIM-Werkbank: Plan, Schnitt, Inspector, Skizzen, Panels (Bauablauf, Berechnungsliste, KV, Mängel, Modul-Editor, Raster)
vis	Visualisierung/Render-Graph: Node-Canvas, HomeStation-Bridge-Bilder, Job-Laufzeit
data	KosmoData: Bauteil-/Materialkatalog, Referenzen, Wissen
publish	Blätter/Plan-Publikation & PDF-Export
prepare	Wissens-Vorbereitung, OneDrive-Import
doc	Dokumentation, Tech-Radar
train	Training/Korpus-Pflege
asset	Asset-Bibliothek, GLB→Mesh, Standbilder
dev	KosmoDev-Workorders (Entwicklungs-Auftragsbuch)

Shell (`src/shell/`): Orbit-Startmenü, Command-Palette, Companion, Kosmo-Orb/-Panel/-Symbol, Einstellungen, Phasen-Leiste. Zustand (`src/state/`) über Zustand-Stores — Laufzeit-Daten strikt getrennt vom synchronisierten Modell.

Test- & Qualitäts-Infrastruktur

- **Vitest-Suiten:** Kernel (35 Dateien, 765 Tests inkl. 28 Golden-SVGs), App (63 Dateien, 847 Tests), AI, Data (mit Datenschutz-Leak-Gate), UI, Contracts.
- **Golden-SVG-QA** (`npm run svg-qa`): rastert jedes Golden in echtem Chromium und prüft Rendering-Validität, viewBox-Fit und Text-Containment (harte Kriterien → Exit-Code).
- **E2E:** 109 Playwright-Specs inkl. voller «Journeys» (ein Haus komplett über den Kosmo-Chat bauen).
- **Sicherheits-Gates:** `secret-scan` (Anthropic/AWS/Entropie/.env) und `netz-check` (Firewall-Konfig-Lint + Bind-Smoke).
- **Release-Gate** (neu 0.7.4): `typecheck && test && svg-qa && security:secrets` in einem Befehl.

CI / Packaging (GitHub Actions)

- **Desktop:** Tauri-Matrix **3 Editionen × 3 Plattformen** (macOS/Linux/Windows).
- **iOS:** Tauri-Experiment (unsigned, best-effort).
- **Pages:** Web/PWA (cloud-Edition) auf GitHub Pages.
- **Build-Request-Mechanik:** eine Zeitstempel-Datei (`.desktop-build-request` etc.) anfassen und pushen löst den Build aus. Keine Signing-Keys → Update = neuer Installer.

BAU-STORY — WIE ES ENTSTAND

KosmoOrbit ist über viele versionierte Grossaufträge «gevibecodet» worden — aber mit einer strengen, ehrlichen Disziplin, nicht als Ad-hoc-Generierung.

Agenten-Orchestrierung (Owner-Guideline, verbindlich)

Ein festes Modell-Regime steuert, wer welche Arbeit macht:

- **Opus orchestriert** — der Leiter zerlegt den Auftrag in Wellen, delegiert und urteilt am Ende; das Spitzenmodell ist nie selbst der Ausführer.
- **Sonnet führt aus** — die eigentliche Bauarbeit (Code, Tests, Design gegen Spec) läuft in Sonnet-Subagenten, je Welle dateidisjunkt.
- **Fable urteilt** — die härtesten 10-15 % (Spec-Freeze, Golden-Verdikte, Schlussreview) gehen an das Urteilsmodell.

«Selbst gevibecodet» heisst hier konkret: nicht generischer Glue-Code, sondern die zod-getriebene Command-Registry und die ~60 puren Derive-Funktionen als Eigenentwicklung — die tragende IP. Einge kaufte Bausteine (React, Three.js, Yjs ...) sind bewusst auf das Nötige beschränkt.

Die Arbeitsgesetze

- **Dateidisjunktheit = Merge-Gesetz:** parallele Agenten fassen nie dieselbe Datei an — so gibt es keine Merge-Konflikte.
- **Golden-Sammelwechsel-Disziplin:** bevor ein Golden-SVG neu erzeugt wird, wird die erwartete Änderung *vorab* als Liste geschrieben und danach Zeile für Zeile maschinell verifiziert — kein Golden ändert sich unbemerkt.
- **Kritik-Runden gegen Soll-Bilder:** jede visuelle Welle wird gegen die vom Owner gelieferten Soll-Vorlagen geprüft (Screenshot-Vergleich), mehrfach bis «angenommen».
- **Push-Early:** nach jedem grünen Schnell-Gate wird sofort gepusht — der Entwicklungs-Container wird gelegentlich zurückgesetzt, die Arbeit ist nur auf dem Remote sicher.
- **Ehrlichkeit vor Politur:** was ein Konto, ein Schlüssel oder die HomeStation braucht, wird im UI offen benannt statt vorgetäuscht — und jede bekannte Grenze steht in den Release-Notizen.

Beispiel 0.7.4 «Einlösen & Feinschliff»

Drei Wellen (Druck-Typografie · Shell/App · neue Features), drei Kritik-Runden, alle angenommen. Eingelöst wurden genau die in 0.7.3 *namentlich* vertagten Punkte (SIA-Hochzahl im PDF, Live-Plan-Phasen, Kosmo-Orb ins Dock, Beschlag-Katalog). Ehrlich offen geblieben und dokumentiert: Lato-400 im PDF-Export, Beschlag-Stufe 2. Gates: Kernel 765/765, App 847/847, svg-qa 28 Goldens / 0 harte Fehler.

Automatisch erzeugt (technik-pdf-074.mts) aus der verifizierten Repo-Bestandsaufnahme, Stand v0.7.4.